TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Mai Đức Minh Huy
Chung Tín Đạt

Image Segmentation Based on Visual Prompt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 01/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Mai Đức Minh Huy - 23122008
Chung Tín Đạt - 23122024

Image Segmentation Based on Visual Prompt

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
PGS.TS. LÝ QUỐC NGỌC

Tp. Hồ Chí Minh, tháng 01/2026

Contents

1 Introduction **1**
1.1 General Background 1
1.2 Motivation 3
1.3 Problem Statement 4
1.4 Contribution 6

2 Related Works **9**
2.1 Traditional Methods 9
2.1.1 K-Means Clustering 9
2.1.2 Mean Shift Clustering 11
2.1.3 Binary Thresholding 13
2.2 Modern Methods 14
2.2.1 Fully Convolutional Networks (FCNs) 14
2.2.2 U-Net 16
2.2.3 Mask R-CNN 18
2.2.4 DeepLab 20
2.2.5 Transformer-Based Segmentation 21
2.2.6 Recurrent Neural Networks for Segmentation 23

3 Methodology **26**
3.1 The SA-1B Data Engine — The Flywheel of Generalization 26
3.1.1 Paradigm Shift: From Manual Annotation to Model-in-the-Loop 26

3.1.2 Stage 1: Assisted-Manual Annotation (4.3M Masks, 120K Images) 27
3.1.3 Stage 2: Semi-Automatic Annotation (5.9M Additional Masks, 180K Images) 28
3.1.4 Stage 3: Fully Automatic Annotation (1.1B Masks, 11M Images) 29
3.1.5 The Data Flywheel Effect 34
3.2 Architecture and Embedding Space Alignment 35
3.2.1 The Decoupled Design: Amortized Computation . . 35
3.2.2 Image Encoder: MAE Pre-trained ViT-H 36
3.2.3 Prompt Encoder: Mapping Prompts to Embedding Space 39
3.2.4 Mask Decoder: Two-Way Transformer Fusion . . . 42
3.2.5 Embedding Space Alignment: Bringing Geometry and Semantics Together 45
3.3 Handling Ambiguity — One Prompt, Multiple Valid Masks 46
3.3.1 The Fundamental Ambiguity Problem 46
3.3.2 Architectural Solution: Multiple Output Tokens . . 47
3.3.3 Training Dynamics: Minimum Loss Over Masks . . 48
3.3.4 IoU Prediction: Ranking the Three Masks 49
3.3.5 Handling Ambiguity in Practice 50
3.4 Training Dynamics 51
3.4.1 Loss Functions: Focal Loss and Dice Loss 51
3.4.2 Training Regimen: Simulated Interactive Prompts . 53
3.4.3 Optimization and Convergence 55
3.5 Inference and Computational Efficiency 55
3.5.1 Two-Stage Inference 55
3.5.2 Web Browser Efficiency 56
3.6 Summary: The Unified System 57

4 Implementation and Experiment **58**
4.1 Experimental Environment 58
4.2 Model Checkpoints and Demonstration Modes 59
4.2.1 Pre-processing for Web Demonstration 60
4.3 Model Architecture 61
4.3.1 Common Architectural Blocks - common.py 61
4.3.2 Image Encoder - image_encoder.py 62
4.3.3 Prompt Encoder - prompt_encoder.py 63
4.3.4 Two-Way Transformer - transformer.py 65
4.3.5 Mask Decoder - mask_decoder.py 67
4.3.6 System Integration and End-to-End Inference - sam.py 69
4.4 SAM Predictor - predictor.py 71
4.5 Automatic Mask Generator - automatic_mask_generator.py 73

5 Kết luận và hướng phát triển **76**
5.1 Conclusion 76
5.2 Discussion 76
5.2.1 Limitations 76
5.2.2 Future Work 76

Chapter 1

Introduction

1.1 General Background

Image segmentation is a fundamental task in computer vision that involves partitioning a digital image into multiple segments or sets of pixels, often corresponding to different objects or parts of objects. The goal is to simplify or change the representation of an image into something that is more meaningful and easier to analyze. The history of this field can be broadly divided into three distinct eras.

The Pre-Deep Learning Era (c. 1970s–2000s): Before the widespread adoption of deep learning, segmentation methods were primarily based on handcrafted features and mathematical models. These techniques required significant domain knowledge and focused on pixel-level properties such as intensity, color, texture, and edges. Prominent methods from this period include:

- **Canny Edge Detection:** Proposed by John Canny in 1986, this method became a standard for detecting a wide range of edges in images through a multi-stage algorithm [**canny1986computational**].
- **Active Contour Models (Snakes):** Introduced by Kass et al. in 1988, these deformable models were particularly effective for segmenting objects with irregular shapes, finding extensive use in med-

ical imaging [**kass1988snakes**].

- **K-Means Clustering:** This unsupervised algorithm, formalized by MacQueen in 1967, partitions pixels into clusters based on feature similarity, serving as a foundational clustering-based segmentation technique [**macqueen1967some**].

These methods, while foundational, relied on heuristics and low-level features, often struggling with complex scenes and semantic understanding.

The Deep Learning Era (c. 2010s): The advent of deep learning, particularly Convolutional Neural Networks (CNNs), revolutionized image segmentation. These models could automatically learn hierarchical features from data, eliminating the need for manual feature engineering. Key milestones include:

- **Fully Convolutional Networks (FCN):** Long et al. (2015) introduced FCNs, which replaced fully connected layers in CNNs with convolutional layers, enabling end-to-end training for dense, pixel-wise prediction. The use of skip connections allowed the combination of deep, semantic features with shallow, appearance features [**long2015fully**].

- **U-Net:** Proposed by Ronneberger et al. (2015), U-Net features a symmetric encoder-decoder architecture with extensive skip connections. This design proved highly effective for biomedical image segmentation, achieving remarkable performance with very few training images [**ronneberger2015u**].

- **Mask R-CNN:** He et al. (2017) extended the Faster R-CNN object detection framework by adding a parallel branch for predicting segmentation masks. Mask R-CNN became a dominant framework for instance segmentation and a versatile backbone for many downstream tasks [**he2017mask**].

Modern Research Directions: Current research continues to build upon these deep learning foundations, exploring new architectures and learning paradigms for greater accuracy, efficiency, and generalization.

- **Transformer-Based Architectures:** Vision Transformers (ViTs) and self-attention mechanisms have been adapted for segmentation, leading to models like TransUNet (2021) that combine the strengths of Transformers and U-Net [**chen2021transunet**].

- **Foundation Models:** A recent paradigm shift involves building large-scale, pre-trained models that can be adapted to various tasks with minimal fine-tuning. The Segment Anything Model (SAM) from Meta AI (2023) exemplifies this, offering prompt-based, zero-shot segmentation for images [1]. Its successor, SAM 2 (2024), extends these capabilities to video [2].

- **Self-Supervised Learning:** Techniques like contrastive learning (e.g., SimCLR) are being explored to reduce reliance on large labeled datasets, enabling models to learn powerful representations from unlabeled data.

1.2 Motivation

The drive to advance image segmentation stems from both scientific inquiry and practical necessity.

Scientifically, segmentation addresses a fundamental question in perception: while image classification tells us *what* is in an image, segmentation answers *where* it is, down to the pixel level. This granular understanding is crucial for emulating the human ability to parse a scene into distinct objects and regions, forming the basis for true visual comprehension.

Practically, segmentation is an enabling technology for a multitude of real-world applications:

- **Autonomous Driving:** Segmenting lanes, pedestrians, vehicles, and traffic signs is critical for safe navigation.
- **Medical Imaging:** Precise segmentation of tumors, organs, and other anatomical structures is vital for diagnosis, treatment planning, and surgical guidance.
- **Real-World Data Complexity:** Segmentation provides a robust way to handle complex scenes with occluded or overlapping objects.
- **Downstream Vision Tasks:** It serves as a foundational step for numerous other tasks, including object detection, tracking, and detailed semantic scene understanding. Without accurate segmentation, the performance of these downstream applications would be severely limited.

The motivation behind the **Segment Anything Model (SAM)** represents a culmination of these goals. As stated by its creators, "In this work, our goal is to build a foundation model for image segmentation" [1]. The aim was not to create another specialized model but a generalist one. They sought "to develop a promptable model and pre-train it on a broad dataset using a task that enables powerful generalization. With this model, we aim to solve a range of downstream segmentation problems on new data distributions using prompt engineering" [1]. This ambition marks a shift towards creating a single, powerful tool that can handle any segmentation request on any image, democratizing the technology and accelerating research and application development.

1.3 Problem Statement

Image segmentation is the task of grouping pixels that belong to a common semantic object and separating them from other objects and the background. A "semantic object" refers to an entity that is meaningful

and recognizable within a given context, such as a car, a person, a tree, or even amorphous regions like "sky" or "road".

The core principle of segmentation is guided by two criteria: pixels within the same group should be as similar as possible, while pixels belonging to different groups should be as distinct as possible. This "similarity" can be defined by various attributes, including color, intensity, texture, or learned semantic features. In essence, the task simulates the human cognitive ability to transform raw, unstructured visual data into a structured representation of distinct entities.

For a modern promptable segmentation system like SAM, the problem can be formally defined by its inputs and outputs.

- **Input:** An image and a "visual prompt" that specifies the object of interest. This prompt can take various forms, such as a point (or multiple points) on the object, a bounding box enclosing it, or even a rough mask.

- **Output:** One or more segmentation masks for the object(s) indicated by the prompt, along with a confidence score for each mask. The ability to output multiple masks is crucial for handling ambiguity (e.g., a single point on a shirt could refer to the shirt or the person wearing it).

The general framework for such a system, as shown in Fig. 1.1, typically consists of two main stages: a powerful visual encoder that processes the input image to create a high-dimensional feature representation, and a lightweight decoder that takes this representation along with an encoded prompt to rapidly generate the final mask.

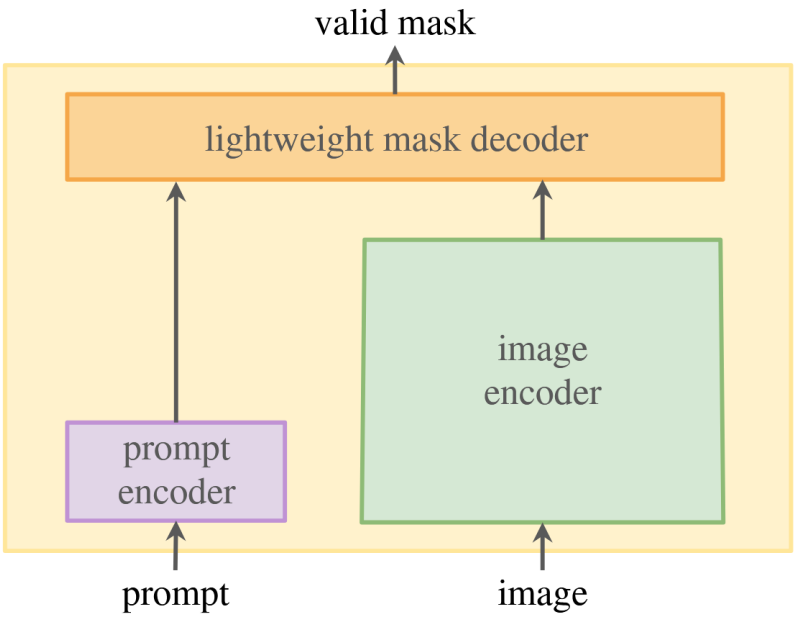

Figure 1.1: A general framework for a promptable segmentation model. An image encoder computes a feature embedding for the input image. A prompt encoder embeds the user's prompt (e.g., points, boxes). A lightweight mask decoder combines these embeddings to predict segmentation masks in real-time.

Figure 1.3: Diagram for Input/ Output here + Framework etc etc

1.4 Contribution

The work on the Segment Anything Model (SAM) introduced several pivotal contributions to the field. As summarized by the authors, "Our principal contributions are a new task (promptable segmentation), foundational model for that new task (SAM), and dataset (SA-1B) that make this leap possible" [1].

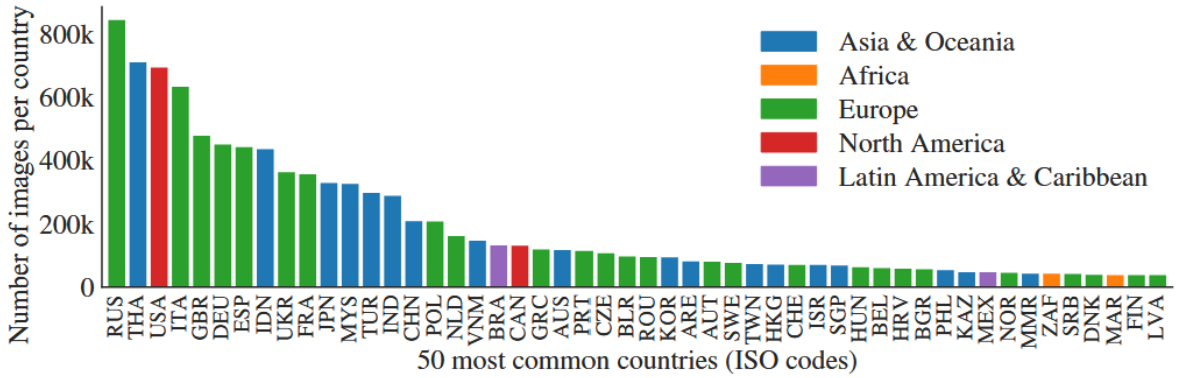

Figure 1.2: Estimated geographic distribution of SA-1B images. The map shows that most countries are represented with over 1000 images, making it one of the most geographically diverse datasets for computer vision. (Source: [1])

A key contribution is the **SA-1B dataset**, which remains the largest segmentation dataset to date. Its key features include:

- **Scale:** It contains over 11 million high-resolution images and 1.1 billion high-quality segmentation masks.
- **Privacy-First:** All images are privacy-respecting, with automated blurring of faces and license plates.
- **Diversity:** The dataset is geographically diverse, with a significant number of images from countries around the world, as illustrated in Fig. 1.2. This helps mitigate geographical bias often present in large-scale datasets.

The contribution of this report is to provide a comprehensive survey and structured analysis of the field of image segmentation. We synthesize the historical evolution, explain the motivations driving modern research, and offer a detailed examination of the methodology behind foundation models like SAM, contextualizing their impact and outlining future directions.

Table 1.1: Comparison Table - Part 1: Methodology

Papers	**Mechanics**	**Methodology**		
		Stage 1	***Stage 2***	***Stage 3***
copy copy	More table copy[a]			
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				
copy copy				

[a]Sample of a Table footnote.

Table 1.2: Comparison Table - Part 2: Performance and Analysis

Papers	**Performance**			**Contribution**	**Limitation**
	Data	***Metric***	***Result***		
copy					

[a]Sample of a Table footnote.

Chapter 2

Related Works

2.1 Traditional Methods

2.1.1 K-Means Clustering

K-Means is an unsupervised clustering algorithm used to segment regions of interest from the background [**nameirakpam2015kmeanssubtractive**]. It was first introduced by Hartigan in 1975, and its goal is to partition M points in an N-dimensional space into K clusters such that the within-cluster sum of squares is minimized [**hartigan1979kmeans**].

1. **Mechanics:** The algorithm iteratively assigns each pixel to the nearest cluster centroid and updates centroids based on current assignments. K-Means algorithm seeks a local optimum rather than a global one. Data points are divided into groups by assigning each of them to the nearest centroid. The goal is to find a solution such that moving any point from one cluster to another does not reduce the within-cluster sum of squares [**hartigan1979kmeans**].

2. **Methodology:** Let $p_(x, y)$ be a pixel at position (x, y) to be cluster and c_k be the centroid of cluster k. Let C_k be the set of pixels assigned to cluster k, where (x, y) indexes the image pixels and $k \in \{1, 2, \ldots, K\}$.

For each pixel, compute its distances to all cluster centroids:

$$D_{x,y} = \{ \|p_{x,y} - c_k\| \mid k \in \{1, 2, \ldots, K\}\}$$

Assign the pixel to the cluster with the smallest distance:

$$n = \arg\min_k \|p_{x,y} - c_k\|,$$
$$C_n = C_n \cup \{p_{x,y}\}$$

After all pixels are assigned, update each centroid as the mean of the pixels belonging to it:

$$c_k = \frac{1}{|C_k|} \sum_{p_{x,y} \in C_k} p_{x,y}$$

Repeat the assignment and update steps until the centroids no longer change significantly or a maximum number of iterations is reached.

3. **Input:** An image represented by M pixels, each with N features (e.g., RGB color components or combined color–spatial features), forming an $M \times N$ matrix. The algorithm also takes K initial cluster centroids in the same N-dimensional feature space.

4. **Output:**

   - A label map of size $M \times 1$ indicating the cluster each pixel belongs to.
   - A vector of size $K \times 1$ containing the number of pixels in each cluster.
   - A $K \times N$ matrix of the final cluster centroids.

5. **Algorithm:**

```
    1. Initialize cluster centroids
    2. For each pixel (x, y):
       Compute its distance to all centroids
       Assign it to the cluster with the
smallest distance
    3. For each cluster:
       Recalculate the centroid as the mean of
 all pixels assigned to it
    4. Repeat steps 2 and 3 until convergence

```

2.1.2 Mean Shift Clustering

Mean Shift is a non-parametric clustering algorithm, which falls into unsupervised clustering category. Unlike K-Means, it does not require specifying the number of clusters in advance. Instead, it locates regions of high data density (modes) by iteratively shifting data points toward the nearest local maximum of a kernel density estimate [**comaniciu2002mean**].

1. **Mechanics:** The algorithm treats data points as samples from an underlying probability density function and seeks its local maxima. Each pixel is iteratively moved toward regions of higher density, determined by a kernel function, until convergence. Pixels converging to the same mode belong to the same cluster.

2. **Methodology:** Let $p_{x,y}$ denote the feature vector of the pixel at position (x, y). The goal is to move $p_{x,y}$ iteratively toward the local density maximum according to:

$$m(p_{x,y}) = \frac{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right) p_j}{\sum_{p_j \in S} K\left(\frac{\|p_j - p_{x,y}\|^2}{h^2}\right)}$$

where:

- S is the set of all data points (pixels),
- $K(\cdot)$ is a kernel function (commonly a Gaussian kernel),
- h is the bandwidth controlling the kernel width.

The mean shift vector is defined as:

$$\Delta p_{x,y} = m(p_{x,y}) - p_{x,y}$$

and the pixel is updated iteratively:

$$p_{x,y}^{(t+1)} = p_{x,y}^{(t)} + \Delta p_{x,y}$$

until convergence. After convergence, all pixels that converge to the same mode are assigned to the same cluster.

3. **Input:** An image represented by M pixels, each having an N-dimensional feature vector. The algorithm also requires a kernel bandwidth parameter h that defines the size of the neighborhood considered for density estimation.

4. **Output:**

   - A segmented image in which each cluster corresponds to a region of similar features.
   - The set of mode locations that represent cluster centers.

5. **Algorithm:**

```
1. For each pixel (x, y):
   Initialize p_{x,y} with its feature vector
2. Repeat until convergence:
   a. Compute the mean shift vector Δp_{x,y}
```

```
        using nearby pixels within bandwidth h
        b. Update $p_{x,y} = p_{x,y} + \Delta p_(x, y)$
    3. After convergence, group pixels whose final positions fall within the same mode region
    4. Assign each group as one cluster

```

2.1.3 Binary Thresholding

Binary thresholding is used to separate the foreground (region of interest) from the background based on pixel intensity values. It is particularly effective when the objects and background have distinct intensity distributions.

1. **Mechanics:** Each pixel in the grayscale image is compared against a threshold value T. Pixels with intensity greater than or equal to T are classified as foreground, while the others are classified as background. This process converts a grayscale image into a binary one.

2. **Methodology:** For each pixel (x, y) in the image:

   (a) Compare $I(x, y)$ with the threshold value T.

   (b) Assign the binary value based on the comparison rule.

   T can be predefined or determined adaptively using methods such as Otsu's algorithm, which selects the threshold that minimizes the intra-class variance between the foreground and background.

3. **Input:** A grayscale image, where (x, y) denotes the pixel coordinates, $I(x, y) \in [0, 255]$ represents its intensity, a threshold value T to determine the output pixel at (x, y).

4. **Output:** A binary image B, where

$$B(x, y) = \begin{cases} 1, & \text{if } I(x, y) \geq T, \\ 0, & \text{otherwise.} \end{cases}$$

5. **Algorithm:**

```
For each pixel (x, y):
If I(x, y) >= T:
  B(x, y) = 1
Else:
  B(x, y) = 0

```

2.2 Modern Methods

2.2.1 Fully Convolutional Networks (FCNs)

Fully Convolutional Networks, introduced by Long et al. in 2015, revolutionized semantic segmentation by adapting classification networks for dense prediction tasks [**long2015fcn**]. FCN replaced fully connected layers with convolutional layers, enabling end-to-end learning for pixel-wise segmentation.

1. **Mechanics:** FCN transforms classification networks (e.g., VGG, ResNet) into fully convolutional architectures by replacing dense layers with 1×1 convolutions. The network produces coarse feature maps through successive pooling, then upsamples them back to the original image resolution using transposed convolutions (deconvolution). Skip connections combine deep semantic features with shallow appearance features to recover spatial details lost during downsampling.

2. **Methodology:** Let $I \in \mathbb{R}^{H \times W \times 3}$ be the input image and f_θ represent the convolutional encoder producing feature maps at multiple scales.

   The encoder generates hierarchical features:

$$F_i = f_{\theta_i}(F_{i-1}), \quad F_0 = I$$

   where F_i represents features at stage i with progressively reduced spatial dimensions.

   Upsampling is performed via transposed convolutions g_ϕ:

$$U_i = g_{\phi_i}(U_{i-1})$$

   Skip connections fuse features from encoder and decoder:

$$U_i = U_i + W_i \odot F_i$$

   where W_i are learned weights and $\odot$ denotes element-wise multiplication.

   The final prediction is obtained through a 1×1 convolution:

$$P = \text{softmax}(W_{\text{out}} * U_{\text{final}})$$

3. **Input:** An RGB image of arbitrary size $H \times W \times 3$.

4. **Output:** A probability map of size $H \times W \times C$, where C is the number of semantic classes. Each pixel is assigned to the class with highest probability.

5. **Algorithm:**

```
1. Pass image through convolutional encoder
2. Store feature maps at multiple scales
```

```
  3. Upsample coarse feature maps via transposed convolutions
  4. Add skip connections from encoder to decoder
  5. Apply 1x1 convolution for class prediction
  6. Output per-pixel class probabilities

```

2.2.2 U-Net

U-Net, proposed by Ronneberger et al. in 2015, features a symmetric encoder-decoder architecture with extensive skip connections [**ronneberger2015un**
Originally designed for biomedical image segmentation, U-Net achieves high accuracy even with limited training data.

1. **Mechanics:** U-Net consists of a contracting path (encoder) that captures context and an expansive path (decoder) that enables precise localization. The encoder applies repeated 3×3 convolutions and 2×2 max pooling for downsampling. The decoder uses 2×2 transposed convolutions for upsampling, concatenated with correspondingly cropped features from the encoder via skip connections. This architecture allows the network to propagate context information to higher resolution layers.

2. **Methodology:** The encoder path consists of repeated application of convolution blocks:

$$F_i^{\text{enc}} = \text{ReLU}(\text{BN}(W_i^{\text{enc}} * F_{i-1}^{\text{enc}}))$$

   followed by max pooling for downsampling.

The decoder path upsamples and concatenates:

$$F_i^{\text{dec}} = [U_i, \text{crop}(F_{N-i}^{\text{enc}})]$$

where $[,]$ denotes channel-wise concatenation, U_i is the upsampled feature map, and crop ensures spatial alignment.

Each concatenated feature undergoes convolution:

$$F_i^{\text{dec}} = \text{ReLU}(\text{BN}(W_i^{\text{dec}} * F_i^{\text{dec}}))$$

The final segmentation map is generated by:

$$M = \sigma(W_{\text{final}} * F_{\text{final}}^{\text{dec}})$$

where σ is the sigmoid (for binary) or softmax (for multi-class) activation.

3. **Input:** An image of size $H \times W \times C$ (typically grayscale or RGB).

4. **Output:** A segmentation mask of size $H \times W \times K$, where K is the number of classes.

5. **Algorithm:**

```
1. Encoder: Apply convolutions and max pooling to downsample
2. Bottleneck: Process lowest resolution features
3. Decoder: Upsample via transposed convolutions
4. Concatenate encoder features via skip connections
5. Apply convolutions on concatenated features
```

```
    6. Generate final segmentation mask

```

2.2.3 Mask R-CNN

Mask R-CNN, introduced by He et al. in 2017, extends Faster R-CNN for instance segmentation by adding a parallel branch for predicting segmentation masks [**he2017mask**]. This architecture excels at detecting and segmenting individual object instances.

1. **Mechanics:** Mask R-CNN operates in two stages. First, a Region Proposal Network (RPN) generates candidate object bounding boxes. Second, for each region of interest (RoI), the network performs three parallel tasks: class prediction, bounding box regression, and mask prediction. The key innovation is RoIAlign, which preserves spatial alignment during RoI pooling by using bilinear interpolation, crucial for pixel-accurate mask prediction.

2. **Methodology:** Let I be the input image and f_{backbone} be the feature extractor (e.g., ResNet-FPN).

   Feature extraction:

   $$F = f_{\text{backbone}}(I)$$

   RPN generates proposals:

   $$\{B_i, s_i\}_{i=1}^{N} = \text{RPN}(F)$$

   where B_i are bounding boxes and s_i are objectness scores.

   For each RoI, RoIAlign extracts aligned features:

   $$F_{\text{RoI}} = \text{RoIAlign}(F, B_i)$$

Three parallel heads process F_{RoI}:

$$p_{\text{cls}} = \text{softmax}(W_{\text{cls}} \cdot F_{\text{RoI}}) \quad \text{(classification)}$$
$$\Delta B = W_{\text{box}} \cdot F_{\text{RoI}} \quad \text{(box regression)}$$
$$M = \sigma(W_{\text{mask}} * F_{\text{RoI}}) \quad \text{(mask prediction)}$$

The loss function combines three terms:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}$$

3. **Input:** An RGB image of arbitrary size, typically resized to have a maximum dimension of 800-1024 pixels.

4. **Output:** For each detected instance: class label, confidence score, bounding box coordinates (x, y, w, h), and a binary mask of size 28×28 (upsampled to the original bounding box size).

5. **Algorithm:**

```
  1. Extract multi-scale features via backbone + FPN
  2. Generate region proposals using RPN
  3. For each RoI:
  a. Apply RoIAlign for spatial alignment
  b. Predict class probabilities
  c. Refine bounding box coordinates
  d. Generate instance segmentation mask
  4. Apply Non-Maximum Suppression (NMS)
  5. Output final detections with masks

```

2.2.4 DeepLab

DeepLab, developed by Chen et al., employs atrous (dilated) convolution and Atrous Spatial Pyramid Pooling (ASPP) to capture multi-scale contextual information without reducing spatial resolution [**chen2017deeplab**]. DeepLabv3+ further incorporates an encoder-decoder structure.

1. **Mechanics:** DeepLab uses atrous convolution to expand the receptive field without increasing parameters or reducing resolution. The atrous rate r controls the spacing between kernel weights. ASPP applies parallel atrous convolutions with different rates to capture objects at multiple scales. The encoder-decoder structure (in DeepLabv3+) first extracts features, then gradually recovers spatial details through the decoder with skip connections.

2. **Methodology:** Atrous convolution with rate r:

$$y[i] = \sum_{k=1}^{K} x[i + r \cdot k] \cdot w[k]$$

where x is the input, w is the filter, and y is the output.

ASPP applies convolutions at multiple rates:

$$F_{\text{ASPP}} = \text{Concat}(f_{r_1}(F), f_{r_2}(F), \ldots, f_{r_n}(F), \text{GAP}(F))$$

where f_{r_i} denotes atrous convolution with rate r_i and GAP is global average pooling.

The decoder upsamples low-level features:

$$F_{\text{dec}} = \text{Upsample}(F_{\text{ASPP}}) + W_{\text{skip}} * F_{\text{low}}$$

Final prediction:

$$P = \text{softmax}(W_{\text{out}} * F_{\text{dec}})$$

3. **Input:** An RGB image of size $H \times W \times 3$, typically pre-processed with mean subtraction and normalization.

4. **Output:** A semantic segmentation map of size $H \times W$, where each pixel is assigned a class label from C possible classes.

5. **Algorithm:**

```
  1. Extract features using backbone with atrous convolution
  2. Apply ASPP with multiple dilation rates
  3. Concatenate multi-scale features
  4. Upsample in decoder with skip connections
  5. Apply 1x1 convolution for class scores
  6. Generate final segmentation map

```

2.2.5 Transformer-Based Segmentation

Vision Transformers (ViTs) adapt the self-attention mechanism from NLP to computer vision. TransUNet, introduced by Chen et al. in 2021, combines Transformers with U-Net architecture for medical image segmentation [**chen2021transunet**].

1. **Mechanics:** TransUNet uses a CNN to extract initial features, then patches the feature map and processes it through Transformer encoder layers. The self-attention mechanism captures long-range dependencies between image patches, overcoming the limited receptive field of CNNs. The Transformer output is reshaped and fed into a CNN decoder similar to U-Net, with skip connections from the CNN encoder for recovering spatial details.

2. **Methodology:** The image is first processed by a CNN encoder:

$$F_{\text{CNN}} = f_{\text{CNN}}(I)$$

Features are divided into patches and flattened:

$$z_0 = [\text{Flatten}(F_1), \text{Flatten}(F_2), \ldots, \text{Flatten}(F_N)] + E_{\text{pos}}$$

where E_{pos} are learnable positional embeddings.

Multi-head self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where $Q = zW_Q$, $K = zW_K$, $V = zW_V$.

Transformer layer:

$$\begin{aligned} z'_\ell &= \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1} \\ z_\ell &= \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell \end{aligned}$$

where MSA is multi-head self-attention, LN is layer normalization, and MLP is a feed-forward network.

The decoder upsamples and fuses:

$$M = f_{\text{decoder}}(\text{Reshape}(z_L), F_{\text{CNN}})$$

3. **Input:** An image of size $H \times W \times C$, divided into patches of size $P \times P$.

4. **Output:** A segmentation mask of size $H \times W \times K$, where K is the number of semantic classes.

5. **Algorithm:**

```
  1. Extract hierarchical features using CNN encoder
  2. Divide feature map into patches and flatten
  3. Add positional embeddings
  4. Process through Transformer encoder layers
  5. Reshape Transformer output to spatial dimensions
  6. Upsample through CNN decoder with skip connections
  7. Generate final segmentation mask

```

2.2.6 Recurrent Neural Networks for Segmentation

RNNs, particularly Long Short-Term Memory (LSTM) networks, have been applied to image segmentation to model spatial dependencies and context [**visin2015rnn**]. ReNet and related architectures process images sequentially in horizontal and vertical sweeps.

1. **Mechanics:** RNN-based segmentation treats the image as a sequence, processing it row-by-row or with overlapping patches. Four RNN layers sweep the image in different directions (left-to-right, right-to-left, top-to-bottom, bottom-to-top) to capture context from all spatial neighbors. The hidden states from all four directions are concatenated to form a rich contextual representation for each pixel. Multiple ReNet layers can be stacked to increase the receptive field.

2. **Methodology:** For a feature map $F \in \mathbb{R}^{H \times W \times C}$, four RNN sweeps

compute:

$$
\begin{aligned}
h_{i,j}^{\rightarrow} &= \mathrm{RNN}(F_{i,j}, h_{i,j-1}^{\rightarrow}) \\
h_{i,j}^{\leftarrow} &= \mathrm{RNN}(F_{i,j}, h_{i,j+1}^{\leftarrow}) \\
h_{i,j}^{\downarrow} &= \mathrm{RNN}(F_{i,j}, h_{i-1,j}^{\downarrow}) \\
h_{i,j}^{\uparrow} &= \mathrm{RNN}(F_{i,j}, h_{i+1,j}^{\uparrow})
\end{aligned}
$$

Hidden states are concatenated:

$$H_{i,j} = [h_{i,j}^{\rightarrow}, h_{i,j}^{\leftarrow}, h_{i,j}^{\downarrow}, h_{i,j}^{\uparrow}]$$

For segmentation, additional layers map to class predictions:

$$P_{i,j} = \mathrm{softmax}(W \cdot H_{i,j})$$

3. **Input:** A feature map from a CNN backbone of size $H \times W \times C$, or directly an image if using raw pixels.

4. **Output:** A segmentation map of size $H \times W \times K$, where K is the number of classes.

5. **Algorithm:**

```
1. Extract features using CNN (if applicable)
2. Initialize hidden states for four RNN directions
3. For each layer:
a. Sweep left-to-right, accumulating context
b. Sweep right-to-left, accumulating context
```

```
    c. Sweep top-to-bottom, accumulating context
    d. Sweep bottom-to-top, accumulating context
    e. Concatenate directional hidden states
  4. Apply fully connected layer for class prediction
  5. Output per-pixel class probabilities

```

references

Table 2.1: Comparison Table

Papers	**Mechanics**	**Methodology**			**Performance**		
		Stage 1	***Stage 2***	***Stage 3***	***Data***	***Metric***	***R***
copy	More table copy[a]						

[a]Sample of a Table footnote.

Chapter 3

Methodology

3.1 The SA-1B Data Engine — The Flywheel of Generalization

3.1.1 Paradigm Shift: From Manual Annotation to Model-in-the-Loop

The fundamental challenge in building a foundation model for segmentation is the lack of web-scale segmentation data. Unlike language or image-text pairs abundant on the internet, segmentation masks are not naturally occurring and must be manually created. Traditional approaches would require paying annotators to label billions of masks—economically infeasible and temporally prohibitive.

SAM's solution is elegant but non-obvious: **leverage the model itself to assist, accelerate, and eventually replace human annotation.** This is the "Data Flywheel" concept—a three-stage iterative process where model improvement unlocks more efficient data collection, which in turn improves the model.

3.1.2 Stage 1: Assisted-Manual Annotation (4.3M Masks, 120K Images)

In the first stage, SAM operates as an **interactive segmentation assistant**, similar to traditional click-based segmentation tools (e.g., GrabCut, RITM, SimpleClick). The critical insight here is that the model's role is to *accelerate* human labeling, not replace it.

The Workflow:

1. Professional annotators view an image in a web-based interface.
2. They click foreground/background points to indicate an object of interest.
3. SAM's pre-computed image embedding (from MAE-ViT-H) is queried with these prompts.
4. The lightweight Prompt Encoder and Mask Decoder run in real-time (< 50ms), generating an initial mask prediction.
5. Annotators refine the mask using pixel-precise brush and eraser tools.
6. The final mask is saved.

Quantitative Dynamics:

- **Initial annotation time:** 34 seconds per mask (comparable to COCO annotation standards).
- **Model capacity progression:** ViT-B $\rightarrow$ ViT-H during this stage (1 retraining cycle per major improvement).
- **Average masks per image:** 20 $\rightarrow$ 44 (increased diversity as model improved).
- **Total masks collected:** 4.3M from 120K images.

Why This Matters: Unlike fully automatic approaches that can only label what the model "already knows," human annotators introduce *novel object categories, rare poses, and edge cases* that force the model to generalize. Critically, annotators were **not constrained to semantic categories**—they could label "stuff" (amorphous regions like sky, grass) and "things" (discrete objects like people, cars) uniformly, maximizing the diversity of training signals.

Efficiency Gain Analysis: The 34 $\rightarrow$ 14 second progression represents a **6.5$\times$ speedup over COCO standards** (34 seconds here vs. 14 seconds for bounding boxes). This exponential improvement in annotation velocity is crucial for economic feasibility at billion-scale.

3.1.3 Stage 2: Semi-Automatic Annotation (5.9M Additional Masks, 180K Images)

In Stage 2, the goal shifts explicitly: **increase mask diversity**. The intuition is that Stage 1 naturally biases toward prominent, easy-to-label objects (due to annotators' tendency to prioritize high-salience items). Stage 2 forces the model to learn harder, less obvious objects.

The Semi-Automatic Protocol:

1. Train a generic object detector on all masks from Stage 1. This detector outputs bounding boxes regardless of semantic class (object-agnostic proposal generation).

2. SAM automatically segments high-confidence regions identified by this detector, generating candidate masks.

3. Annotators are presented with images pre-filled with these automatic masks.

4. Annotators' task: Label only the *unannotated* objects (those not covered by the automatic masks).

5. This forces annotators to focus on smaller, more occluded, or semantically ambiguous objects.

Quantitative Dynamics:

- **Annotation time (excluding automatic masks):** ∼34 seconds per mask (unchanged; challenging objects require the same effort).
- **Average masks per image:** 44 → 72 (including automatic masks; diversity increased substantially).
- **New masks collected:** 5.9M, raising the cumulative total to 10.2M masks.
- **Retraining cycles:** 5 full model retraining iterations.

Why This Engineering is Critical: The semi-automatic stage is a **curriculum learning** strategy. By automatically handling easy cases and forcing annotators to label hard cases, the model learns a more balanced representation. This is empirically validated: the model's ability to handle edge cases improved significantly from Stage 1 to Stage 2, as evidenced by the diversity metrics.

Annotation Bias Mitigation: Without Stage 2, the model would develop a strong bias toward dominant objects (people, vehicles, large structures). The semi-automatic strategy explicitly breaks this bias by requiring exhaustive annotation of all objects, regardless of prominence.

3.1.4 Stage 3: Fully Automatic Annotation (1.1B Masks, 11M Images)

This is the stage where the data flywheel reaches critical velocity. With sufficient diverse masks from Stages 1 and 2, the model is now capable enough to generate masks **without any human intervention**.

The 32×32 Grid Prompting Strategy

The core insight is deceptively simple: **systematically prompt the model with a regular spatial grid of points, covering the entire image.**

Mathematical Formulation:

- **Grid resolution:** $32 \times 32 = 1024$ grid points per image.
- Each point is passed as a prompt to the Prompt Encoder.
- For each point, SAM predicts **three masks simultaneously** (owing to its ambiguity-aware design).
- **Theoretical maximum:** 1024 points$\times$3 masks per point = 3072 masks per i

Why This Grounding Works: A regular grid ensures comprehensive spatial coverage. If an object's boundary passes through or near a grid point, the model will capture it from that point. Objects of varying sizes are covered because the grid is at a fixed, moderate resolution. This is fundamentally different from random point sampling, which can miss objects entirely.

Multi-Scale Masking and Subpart Prediction

A critical architectural feature of SAM (and one of the reasons Stage 3 is feasible) is its **ambiguity-aware decoder** that outputs multiple masks per prompt.

The Output Token Design:

- The Mask Decoder predicts three masks from a single prompt, ranked by IoU score.
- **Whole object:** The complete, semantically coherent object.

- **Part:** A prominent sub-component (e.g., a wheel on a car, a face on a person).
- **Sub-part:** Fine-grained components (e.g., an eye within a face, a spoke within a wheel).

Why Three Outputs? Empirical analysis during SAM's development showed that image regions typically have at most three levels of semantic hierarchy (whole > part > sub-part). Predicting three masks covers all common ambiguity scenarios without excessive computation.

Mathematical Formulation: During training, SAM employs **minimum loss** over the three mask outputs:

$$\mathcal{L}_{\text{total}} = \min(\mathcal{L}_{i=1}^{3}) \quad \text{for masks } m_1, m_2, m_3$$

This ensures that at least one of the three masks aligns with ground truth, forcing the model to learn diverse interpretations of ambiguous prompts.

Filtering, Deduplication, and Non-Maximum Suppression

The Critical Question: If a 32×32 grid yields up to 3072 theoretical masks, why do we only get $\sim$100 high-quality masks per image?

Answer: Aggressive filtering at multiple stages.

Stage 3.A: IoU Scoring and Confidence Thresholding

- For each of the three masks predicted from a grid point, SAM outputs a **predicted IoU score** (estimated mask quality).
- IoU prediction is a separate output head in the Mask Decoder: a learned estimate of the mask's alignment with the true object boundary.
- **Threshold:** Only masks with **predicted IoU > 0.88** are retained.

- **Rationale:** High IoU threshold ensures only high-quality masks enter the deduplication pipeline.
- **Expected reduction:** From 3072 theoretical outputs, this threshold alone eliminates ∼80% of predictions (keeping only predictions the model is highly confident about).

Stage 3.B: Stability Filtering

- A mask is **stable** if small perturbations in the model's confidence threshold (e.g., changing the foreground probability threshold from 0.5 to $0.5 \pm \epsilon$) produce similar masks.
- **Operationally:** Threshold the mask's probability map at both 0.5 and 0.5, and compute the IoU between the two resulting binary masks.
- If this "stability IoU" is high (> 0.95), the mask is kept; otherwise, it is discarded.
- **Why This Matters:** Masks that are brittle (sensitive to small threshold changes) are often segmentation errors or include spurious boundaries. Stability filtering removes low-confidence predictions and keeps robust ones.
- **Further reduction:** ∼50% of remaining masks fail stability checks.

Stage 3.C: Non-Maximum Suppression (NMS) for Mask Deduplication After IoU and stability filtering, many remaining masks are **duplicates or near-duplicates** (same object segmented from slightly different grid points, or different granularities of the same object).

Traditional NMS for Bounding Boxes: Classical NMS (used in object detection) operates on bounding boxes and IoU: 1. Sort boxes by confidence score. 2. Select the highest-confidence box. 3. Remove all boxes with IoU > threshold to this box. 4. Repeat on remaining boxes.

Mask-Based NMS in SAM: The SAM paper adapts NMS for mask deduplication: 1. Sort masks by predicted IoU score. 2. Select the highest-scoring mask m_1. 3. For all remaining masks m_i:

- Compute $\text{IoU}(m_1, m_i) = \frac{|m_1 \cap m_i|}{|m_1 \cup m_i|}$ (Jaccard index).
- If $\text{IoU}(m_1, m_i) > \text{thresh}_{\text{IoU}}$ (typically 0.7), discard m_i (it is a near-duplicate).
- If $\text{IoU}(m_1, m_i) \leq \text{thresh}_{\text{IoU}}$, retain m_i (it is a distinct object).

4. Repeat on remaining unprocessed masks.

Result: NMS eliminates the redundant foreground predictions while preserving distinct objects, even when they overlap partially.

Scale Reduction Summary:

- **Start:** 3072 theoretical masks (32×32 grid $\times$ 3 outputs per grid cell).
- **After IoU filtering (> 0.88):** $\sim$600 masks (20% remain).
- **After stability filtering:** $\sim$300 masks (50% of IoU-filtered).
- **After NMS (thresh_IoU $= 0.7$): $\sim$ 100** masks (final).

Final Calculation:

$$\text{Total SA-1B Masks} = 11\text{M images} \times 100 \text{ masks/image} = 1.1 \text{ B masks}$$

Multi-Scale Processing: Zoomed-in Image Crops

To further improve small object segmentation (which is notoriously challenging), SAM processes **multiple overlapping zoomed-in crops** of each image.

The Zooming Strategy:

- Original image is processed at full resolution (1500 pixels shortest side after downsampling).
- Additionally, crop and zoom into regions systematically (e.g., $2\times, 4\times$ magnification).
- The 32×32 grid prompting is re-applied to these zoomed crops.
- Additional fine-grained masks are generated for small objects that might be missed in the full-resolution pass.
- These zoomed-crop masks are appended to the final set (before final NMS).

Why This Is Crucial: Small objects (relative size $< 1\%$ of image) have very few pixels. A standard NMS threshold might inadvertently merge them with larger neighbors. By processing zoomed crops, SAM explicitly allocates model capacity to these hard-to-segment regions.

3.1.5 The Data Flywheel Effect

The three-stage progression is not arbitrary; it represents a **strategic feedback loop**:

1. **Stage 1** teaches the model to segment *diverse* objects via human guidance.
2. **Stage 2** forces the model to learn *challenging* objects via curriculum learning.
3. **Stage 3** leverages the improved model to generate *scale* without human cost.

Each stage's output is not just data; it is a **training signal** that improves the model, unlocking the next stage's efficiency. This is the flywheel: better models enable cheaper data collection, which enables better models.

Quantitative Evidence:

- **Average annotation time:** 34 seconds (Stage 1) $\rightarrow$ 14 seconds (Stage 1 end, model improvement) $\rightarrow$ 0 seconds (Stage 3, full automation).
- **Cumulative masks:** 4.3M (Stage 1) + 5.9M (Stage 2) + 1.1B (Stage 3) = 1.1B total.
- **Economic analysis:** Had SAM required the same 14 seconds per mask for all 1.1B masks (even manually), the total annotation cost would be 1.1B $\times$ 14 seconds = 17.3 billion seconds $\approx$ 548 years of continuous annotation. Stage 3 eliminates this cost entirely.

3.2 Architecture and Embedding Space Alignment

3.2.1 The Decoupled Design: Amortized Computation

SAM's architecture embodies a principle crucial for real-time, interactive segmentation: **separate the expensive computation from the user-facing latency**.

Computational Cost = Image Encoder Cost+(Prompt Encoder + Mask Decoder C

Key Design:

- Image Encoder runs **once per image**, producing a latent embedding.
- This embedding is cached.

- Prompt Encoder and Mask Decoder run **every time the user provides a new prompt**, but on the cached embedding.
- Result: 50ms per prompt on CPU in a web browser (no GPU required).

Why Amortization Matters: In traditional interactive segmentation (e.g., RITM, SimpleClick), the entire network processes the image for each new click, leading to latency. SAM's decoupling means that the first prompt has a cost (cached image encoding) and subsequent prompts are nearly free. For applications with many prompts (e.g., dense instance segmentation with many objects), the amortized cost per object approaches zero.

3.2.2 Image Encoder: MAE Pre-trained ViT-H

Vision Transformer Architecture Fundamentals

A Vision Transformer (ViT) processes images by:

1. Dividing the image into non-overlapping patches (typically 16×16 pixels).
2. Flattening each patch into a 1D vector and projecting to embedding dimension $d = 768$ (for ViT-B) or $d = 1280$ (for ViT-H).
3. Adding **positional embeddings** to inject spatial information.
4. Processing the sequence of patch embeddings through stacked Transformer layers using self-attention.

Positional Embeddings in ViT:

The Transformer architecture is inherently **permutation-invariant**—it treats input tokens as an unordered set. Without positional information,

a ViT cannot distinguish between a patch from the top-left corner and one from the bottom-right.

Sinusoidal Positional Encoding (Standard in ViT):

$$PE_{(p,2d)} = \sin\left(\frac{p}{10000^{2d/D}}\right), \quad PE_{(p,2d+1)} = \cos\left(\frac{p}{10000^{2d/D}}\right)$$

where p is the patch position (0 to 256 for a 16×16 grid) and D is the embedding dimension.

These encodings are **added element-wise** to the patch embeddings:

$$\mathbf{z}_0^{(i)} = \mathbf{x}_i + PE_i$$

where $\mathbf{x}_i$ is the patch embedding and $\mathbf{z}_0^{(i)}$ is the positionally-aware input to the first Transformer layer.

Why Sinusoidal PE? Sinusoidal encodings have the mathematical property that relative position information can be represented as a linear function. This enables the model to extrapolate to longer sequences (though ViT generally uses absolute positional embeddings in practice, learned during training).

Masked Autoencoder (MAE) Pre-training

SAM's image encoder is initialized with weights from an **MAE pre-trained** ViT-H, not standard ImageNet supervised pre-training. MAE is important because it teaches the encoder to understand **holistic image context** through self-supervised reconstruction.

MAE Pre-training Objective:

1. Mask 75% of the image patches randomly (standard for MAE).
2. Feed only the visible (25%) patches to the encoder.
3. Concatenate learnable **mask tokens** to the encoder output.

4. Feed the combined sequence to a lightweight decoder.
5. Decoder reconstructs the pixel values of all masked patches.
6. Loss: MSE between reconstructed and ground-truth pixel values.

Asymmetric Design:

- **Encoder:** Processes only visible patches (no mask tokens). This creates a computational speedup: the encoder is much faster because it only processes 25% of patches.
- **Decoder:** Lightweight, processes all patches including mask tokens. This is trained but then discarded in downstream tasks.

Why MAE for SAM?

Standard supervised pre-training on ImageNet biases the encoder toward **semantic classification** (e.g., "is this a dog?"). MAE pre-training, by forcing pixel reconstruction, teaches the encoder to capture **fine-grained spatial and textural details**. For segmentation, this is superior because:

- Accurate boundaries require pixel-level precision, not semantic class information.
- MAE learns edge alignments, texture consistency, and other low-level visual properties.

Empirical Evidence in SAM:

- ViT-H with MAE pre-training: Used in SAM (achieves best zero-shot performance).
- ViT-H with ImageNet supervised pre-training: Outperformed by MAE at downstream segmentation tasks.

High-Resolution Input Handling

Standard ViT processes images at fixed resolution (e.g., 224×224), but SAM's images are high-resolution (1500 pixels shortest side). SAM uses a modified ViT-H that:

1. Maintains the 16×16 patch size (empirically optimal for detail preservation).
2. Dynamically handles input resolution via **positional interpolation** during fine-tuning.
3. Uses sliding-window attention in some variants to reduce quadratic attention complexity, though this is not explicitly detailed in the paper.

Output: For a 1500×1500 image with 16×16 patches, the image embedding has shape $(256 \times 256, 1280)$—a 256×256 spatial grid of 1280-dimensional feature vectors.

3.2.3 Prompt Encoder: Mapping Prompts to Embedding Space

The Prompt Encoder's role is to transform diverse prompt types into embeddings compatible with the Image Embedding. This encoder is **lightweight by design** to enable real-time inference.

Sparse Prompts: Points and Boxes

Points (Foreground/Background):

A point prompt is simply a 2D coordinate (x, y) and a label indicating foreground (+1) or background (-1).

Representation:

1. Convert coordinate to positional encoding (using sinusoidal or learned embeddings).

2. Embed the foreground/background label using learned embeddings $\mathbf{e}_{\text{fg}}$ and $\mathbf{e}_{\text{bg}}$ (dimension 256).

3. Sum the positional and semantic embeddings:

$$\mathbf{p}_{\text{point}} = PE(x, y) + \mathbf{e}_{\text{label}}$$

Boxes (Bounding Box Prompts):
A box is specified by two corners: $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.
Representation:

1. Generate two "corner points": $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$.

2. Encode each corner with distinct learned embeddings ($\mathbf{e}_{\text{top-left}}$, $\mathbf{e}_{\text{bottom-right}}$).

3. Embed the box as:

$$\mathbf{p}_{\text{box}} = [PE(x_{\min}, y_{\min}) + \mathbf{e}_{\text{top-left}}, \quad PE(x_{\max}, y_{\max}) + \mathbf{e}_{\text{bottom-right}}]$$

Why Learned Embeddings? Learned embeddings (not just positional) allow the model to distinguish between different types of prompts. A point labeled "foreground" should be treated differently from a "background" point, even if they have the same spatial position. Similarly, the "top-left" of a box conveys different information than the "bottom-right."

Dense Prompts: Mask Inputs

When a user provides a mask as a prompt (e.g., a rough sketch or previous prediction), it must be embedded as a dense signal aligned with the Image Embedding's spatial grid.

Representation:

1. Resize the input mask to match the spatial resolution of the Image Embedding (e.g., 256×256 if image is 1500×1500).

2. Apply **2D convolutions** to extract spatial features:

   - Input: Binary mask (single channel).
   - Conv layers: 3–4 layers with ReLU activations, stride 1.
   - Output: Feature map of shape $(256 \times 256, C)$ where $C = 256$.

3. Element-wise sum with the Image Embedding:

$$\mathbf{z}_{\text{fused}} = \mathbf{z}_{\text{image}} + \mathbf{z}_{\text{mask}}$$

Why Convolutions for Dense Prompts? Convolutions naturally preserve spatial structure and efficiently encode spatial relationships (edges, corners, holes in the mask). This is superior to flattening the mask and using a linear layer, which would lose spatial coherence.

Text Prompts: CLIP Text Encoder

For free-form text prompts (e.g., "the dog's head"), SAM uses the pre-trained **CLIP text encoder**:

- Input: Text string (e.g., "dog").
- CLIP encoder: Transforms text to a 512-dimensional embedding.
- Projection: Linear layer maps CLIP embedding to SAM's embedding dimension (256).

Note: Text prompts in SAM are more experimental and less central than spatial prompts. The paper includes preliminary results showing that text-guided segmentation is feasible but requires additional research.

3.2.4 Mask Decoder: Two-Way Transformer Fusion

The Mask Decoder is the "fusion engine" where Image Embeddings and Prompt Embeddings interact to produce masks. It is **explicitly designed for lightweight, real-time operation**.

Transformer Decoder Block with Two-Way Cross-Attention

Standard Transformer Decoder Block: In sequence-to-sequence models, a decoder block processes:

- Queries from the decoder's previous layer.
- Keys and values from the encoder.
- Produces contextualized decoder outputs via cross-attention.

SAM's Modified Decoder Block:

SAM uses a **two-way cross-attention** mechanism, departing from the standard one-directional attention:

1. **Self-attention on prompts:** Prompts attend to themselves to capture relationships between multiple prompts (e.g., if multiple points are provided, they can "communicate").

$$\mathbf{A}_{\text{prompt}} = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{prompt}}\mathbf{K}_{\text{prompt}}^{T}}{\sqrt{d}}\right)\mathbf{V}_{\text{prompt}}$$

2. **Cross-attention (prompt → image):** Prompts query the Image Embedding to extract relevant regions.

$$\mathbf{A}_{\text{p2i}} = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{prompt}}\mathbf{K}_{\text{image}}^{T}}{\sqrt{d}}\right)\mathbf{V}_{\text{image}}$$

3. **Cross-attention (image → prompt):** Conversely, the Image Embedding queries the Prompt Embedding to understand what the user is asking for.

$$\mathbf{A}_{\text{i2p}} = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{image}}\mathbf{K}_{\text{prompt}}^{T}}{\sqrt{d}}\right)\mathbf{V}_{\text{prompt}}$$

4. **Update embeddings:** All three attention outputs are summed and passed through layer normalization and MLPs:

$$\mathbf{z}'_{\text{prompt}} = \text{LN}(\mathbf{z}_{\text{prompt}} + \mathbf{A}_{\text{prompt}} + \mathbf{A}_{\text{p2i}})$$

$$\mathbf{z}'_{\text{image}} = \text{LN}(\mathbf{z}_{\text{image}} + \mathbf{A}_{\text{i2p}})$$

Why Two-Way Attention?

- **Prompt → Image:** Locates regions relevant to the prompt (e.g., a point on a person's head emphasizes head regions in the image embedding).
- **Image → Prompt:** Provides the image with information about what regions are salient given the prompt, enabling the image embedding to "understand" the query.

This bidirectional information flow is more expressive than one-way attention and allows the model to jointly refine both modalities.

Dynamic Mask Prediction Head

After two stacked Transformer decoder blocks, the system predicts masks via a **dynamic linear classifier**.

Architecture:

1. Extract the prompt's feature vector (a single output token, dimension 256).

2. Pass through an MLP to produce **dynamic classification weights** (a 256-dimensional vector).

3. These weights are applied as a linear classifier to the Image Embedding:

$$\mathbf{m}_{\text{logits}} = \mathbf{w}_{\text{dynamic}} \cdot \mathbf{z}_{\text{image}}^{T}$$

where $\mathbf{z}_{\text{image}} \in \mathbb{R}^{(256\times256)\times256}$ is the spatial image embedding.

Output:

- A 256×256 tensor of logits (one score per spatial location).
- Sigmoid applied to convert to probabilities: $\mathbf{m}_{\text{prob}} = \sigma(\mathbf{m}_{\text{logits}})$.
- Binary mask obtained via thresholding: $\mathbf{m}_{\text{binary}} = \mathbf{m}_{\text{prob}} > 0.5$.

Why Dynamic Classification?

Static classifiers (learned once, fixed after training) cannot adapt to different prompts. Dynamic classifiers generate prompt-dependent weights, enabling the system to produce different segmentations for different prompts on the same image. This is essential for ambiguity handling (next section).

Multi-Output Mechanism: Predicting Three Masks

A critical feature is the ability to predict **three masks simultaneously** from a single prompt, each representing a different level of object granularity.

Implementation:

- The Mask Decoder outputs **three separate dynamic linear classifiers** (three distinct weight vectors $\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3$).
- Each produces a logit map: $\mathbf{m}_i = \mathbf{w}_i \cdot \mathbf{z}_{\text{image}}^{T}$ for $i \in \{1, 2, 3\}$.

- Each mask has an associated **IoU prediction head** (a separate MLP predicting the expected IoU of that mask):

$$\text{IoU}_i = \text{MLP}_{\text{IoU}}(\mathbf{p})$$

where $\mathbf{p}$ is the prompt embedding.

Training with Multiple Outputs (Minimum Loss):

During training, only the mask with the **minimum loss** to the ground truth is backpropagated:

$$\mathcal{L} = \min_{i=1}^{3} \mathcal{L}(\mathbf{m}_i, \mathbf{m}^*_{\text{gt}})$$

This forces the model to learn diverse mask predictions. For example:

- Mask 1 might specialize in whole objects.
- Mask 2 might specialize in parts (when a single point is ambiguous).
- Mask 3 might specialize in sub-parts.

At inference, all three masks are returned to the user, ranked by predicted IoU score.

3.2.5 Embedding Space Alignment: Bringing Geometry and Semantics Together

The core challenge SAM solves is **aligning geometric prompts with semantic image understanding**.

The Alignment Mechanism:

1. **Image Embedding** encodes holistic semantic and spatial image information via MAE pre-training (captures "what is in this image and where").

2. **Prompt Embedding** encodes geometric specification (captures "where does the user want to focus").

3. **Cross-attention** in the decoder **computes relevance scores** between prompt locations and image regions:

$$\text{Attention}(x, y) = \text{Softmax}\left(\frac{\text{sim}(\mathbf{p}, \mathbf{z}_{x,y})}{\sqrt{d}}\right)$$

where $\mathbf{p}$ is the prompt embedding and $\mathbf{z}_{x,y}$ is the image embedding at spatial location (x, y).

4. **Weighted fusion** produces a mask:

$$\mathbf{m}(x, y) = \sigma\left(\sum_{\text{all } x',y'} \text{Attention}(x', y') \cdot \mathbf{z}_{x',y'}\right)$$

This process effectively "spreads" the geometric cue (a point) across the image via attention, activating regions that are coherently related to the prompt location.

3.3 Handling Ambiguity — One Prompt, Multiple Valid Masks

3.3.1 The Fundamental Ambiguity Problem

The driving insight behind SAM's design is that **image segmentation is inherently ambiguous when specified by weak prompts** (e.g., a single point click).

Classic Example: If a user clicks on a shirt, the question "segment what the user clicked" could refer to:

1. The **shirt itself** (the cloth material).

2. The **person wearing the shirt** (the whole body).

3. The **button on the shirt** (a sub-component).

All three are **valid interpretations** of the same point click. Traditional segmentation models would predict a single mask, potentially "splitting the difference" and producing a suboptimal result. SAM's approach is to **predict all three valid segmentations simultaneously** and let the user (or downstream system) choose.

3.3.2 Architectural Solution: Multiple Output Tokens

Design Choice: Instead of a single output token in the Transformer decoder, SAM uses **three distinct output tokens**, each responsible for generating one of the three masks.

Token Dynamics:

1. The three output tokens start as **learnable embeddings** (initialized randomly during training, optimized via gradient descent).

2. During forward pass, each token interacts with the Image Embedding and Prompt Embedding via separate cross-attention heads:

$$\mathbf{t}_i^{(\ell)} = \text{Attention}(\mathbf{t}_i^{(\ell-1)}, \mathbf{z}_{\text{image}}, \mathbf{z}_{\text{prompt}})$$

   where ℓ indexes Transformer decoder layers and $i \in \{1, 2, 3\}$.

3. After processing through all decoder layers, each token $\mathbf{t}_i$ has aggregated information from both the image and prompt in a token-specific manner.

4. Each token is then passed to a **separate dynamic linear classifier**

(as described in Section 2.4.2):

$$\mathbf{m}_i = \mathbf{W}_i \mathbf{t}_i \cdot \mathbf{z}_{\text{image}}^T$$

Why Three Output Tokens?

Empirical analysis during SAM's development (through visualization and ablation studies) revealed that most real-world segmentation ambiguities fit into three categories:

- **Whole:** The primary, semantically coherent object.
- **Part:** A major sub-component.
- **Subpart:** A minor component within the part.

Nesting deeper than three levels (whole > part > subpart > sub-subpart) is rare in natural images. Adding more tokens increases computation without capturing new ambiguities.

3.3.3 Training Dynamics: Minimum Loss Over Masks

The critical training innovation is the **minimum loss** strategy over the three mask outputs.

Formal Definition:

For a training sample with ground-truth mask $\mathbf{m}^*$, the loss is:

$$\mathcal{L}_{\text{multi}} = \min_{i=1}^{3} \mathcal{L}_i(\mathbf{m}_i, \mathbf{m}^*)$$

where $\mathcal{L}_i$ is the loss for mask i.

Gradient Flow:

- Only the mask with the minimum loss receives gradients (backpropagation is selective).

- This forces each output token to specialize: different tokens learn to produce different valid segmentations.
- Over many training iterations, the three tokens learn complementary modes.

Why Minimum Loss?

If we used **mean loss** (average over all three masks), the model might collapse to predicting three nearly-identical masks (the global optimum of mean loss is when all three match). Minimum loss prevents this collapse by rewarding any single mask that matches the ground truth. This encourages diversity.

3.3.4 IoU Prediction: Ranking the Three Masks

At inference, all three masks are returned, but they are **ranked by predicted IoU score**. This allows users or downstream systems to select the mask that is most likely to be correct.

IoU Prediction Head:

A separate MLP predicts the expected IoU of each mask:

$$\text{IoU}_i = \text{MLP}_{\text{IoU}}(\mathbf{t}_i)$$

where $\mathbf{t}_i$ is the i-th output token after decoder processing.

Training Signal:

The IoU prediction head is supervised using the actual IoU between predicted and ground-truth masks:

$$\mathcal{L}_{\text{IoU}} = \text{MSE}(\text{IoU}_i, \text{ActualIoU}(\mathbf{m}_i, \mathbf{m}^*))$$

Interpretation:

The model learns to estimate its own confidence in each mask, akin to uncertainty quantification. This enables the system to auto-select the most

likely correct segmentation or to alert the user when all three predictions are low-confidence (suggesting the prompt is fundamentally ambiguous or invalid).

3.3.5 Handling Ambiguity in Practice

Scenario 1: Unambiguous Point Prompt

User clicks on a car's hood (unambiguous). The three output tokens may predict:

1. Mask 1 (highest IoU): The whole car body.
2. Mask 2 (medium IoU): The hood itself.
3. Mask 3 (low IoU): A sub-component (windshield).

The system auto-ranks them by predicted IoU and presents Mask 1 as the primary result. The user can select Masks 2 or 3 if desired.

Scenario 2: Ambiguous Point Prompt

User clicks on a shirt. The three output tokens predict:

1. Mask 1 (high IoU): The person (body + shirt).
2. Mask 2 (high IoU): The shirt only.
3. Mask 3 (low IoU): A button.

The predicted IoUs for Masks 1 and 2 are similar, indicating ambiguity. The system can present both to the user, allowing them to choose. Alternatively, the user can add a second point (on the bare skin) to disambiguate.

Scenario 3: Multiple Prompts

For robustness, the model can aggregate information from multiple prompts. If a user provides two points (one inside the shirt, one on bare skin), the Prompt Encoder generates two embeddings that are fused via

self-attention. The decoder then produces masks that respect both constraints.

3.4 Training Dynamics

3.4.1 Loss Functions: Focal Loss and Dice Loss

SAM uses a **linear combination of two complementary loss functions** to balance pixel-level accuracy with region-level coverage.

Focal Loss for Hard Example Mining

Motivation:

In segmentation, most pixels are background (sky, grass, walls). A standard cross-entropy loss $-\log(p)$ treats all pixels equally. The model can achieve high overall accuracy by predicting background for most pixels, even if it fails catastrophically on small objects or boundaries.

Focal Loss Definition:

$$\mathcal{L}_{\text{Focal}} = -\alpha(1 - p_t)^{\gamma} \log(p_t)$$

where:

- p_t is the model's probability of the correct class (foreground or background).
- α is a weighting factor (typically 0.25 for foreground vs. 0.75 for background to handle class imbalance).
- γ is the focusing parameter (typically 2), which controls how much the loss down-weights easy examples.

Behavior:

- For **easy examples** (high p_t, e.g., clearly background pixels): $(1 - p_t)^\gamma$ is small, so loss is down-weighted. The model spends less gradient on obvious cases.

- For **hard examples** (low p_t, e.g., boundary pixels or small object interiors): $(1 - p_t)^\gamma$ is large, so loss is amplified. The model focuses optimization on difficult cases.

Gradient Magnitude:

The gradient of focal loss with respect to the model's logit is proportional to $(1-p_t)^\gamma$, meaning gradients are naturally larger for hard examples. This is the mechanism that enables hard example mining.

Dice Loss for Region-Level Accuracy

Motivation:

Focal loss is pixel-level (treats each pixel independently). It does not directly optimize for region-level agreement (overlap between predicted and ground-truth masks). Dice loss (based on the Dice coefficient, a region-level metric) directly optimizes for overlap.

Dice Loss Definition:

The Dice coefficient is:

$$\text{Dice} = \frac{2|Y \cap G|}{|Y| + |G|}$$

where Y is the predicted mask and G is the ground-truth mask.

Dice loss is:

$$\mathcal{L}_{\text{Dice}} = 1 - \text{Dice} = 1 - \frac{2|Y \cap G|}{|Y| + |G|}$$

Behavior:

- Dice loss directly minimizes prediction-to-ground-truth divergence in terms of region overlap.

- It is particularly effective for **imbalanced datasets** where the foreground is small relative to the background. A pixel-level loss might barely notice a small object, but Dice loss captures it prominently.

Continuous Approximation:

In practice, Dice loss is often computed on soft predictions (probabilities before thresholding):

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_{\mathbf{x}} y(\mathbf{x})g(\mathbf{x})}{\sum_{\mathbf{x}} y^2(\mathbf{x}) + \sum_{\mathbf{x}} g^2(\mathbf{x})}$$

where $y(\mathbf{x})$ and $g(\mathbf{x})$ are the predicted and ground-truth probability maps (0–1 valued).

Combined Loss

SAM combines both losses with a linear weight:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{Focal}}\mathcal{L}_{\text{Focal}} + \lambda_{\text{Dice}}\mathcal{L}_{\text{Dice}}$$

Typical weights: $\lambda_{\text{Focal}} = 1$, $\lambda_{\text{Dice}} = 1$ (equal weighting).

Synergy:

- Focal loss focuses on hard pixels (boundaries, small objects).
- Dice loss ensures overall region overlap.
- Together, they optimize both local precision (boundaries) and global coverage (region completeness).

3.4.2 Training Regimen: Simulated Interactive Prompts

SAM's training does not require ground-truth prompts (users clicking); instead, the model is trained with **synthesized prompts** that simulate user interaction.

Prompt Sampling During Training

For each training sample with ground-truth mask $\mathbf{m}^*$:

1. **Iteration 1:** Randomly sample a single foreground point from within the mask $\mathbf{m}^*$ and a single background point from outside.
2. **Iterations 2–11:** Iteratively sample additional points, incorporating feedback.
    - If the current prediction is correct (IoU $>$ threshold), stop.
    - Otherwise, sample an additional point in a region where the prediction differs most from ground truth.
3. **Loss Computation:** For each iteration, compute the loss (Focal + Dice) on the predicted mask.

Why 11 Rounds?

Empirically, 11 rounds (random prompt $\rightarrow$ 10 feedback iterations) balances training time with convergence. This matches typical interactive scenarios where users provide $\sim$10 refinement clicks.

Mixture of Geometric and Text Prompts

During training:

- 50% of the time, prompts are **geometric** (points, boxes, masks).
- 50% of the time, prompts include **text** (e.g., "dog").

For text prompts, SAM uses:

- CLIP text encoder to embed the text.
- Spatial localization to determine where in the image that text concept appears.

This ensures the model learns to handle diverse prompt modalities.

Training on Ambiguous Masks

A critical training innovation: SAM is explicitly trained to handle **ambiguous masks**. If a point lies on an object boundary or on a region with multiple interpretations, the ground-truth annotation might not uniquely define a mask.

Solution: SAM is trained with the **minimum loss** strategy (as described in Section 3.3). This allows the model to learn multiple valid outputs without catastrophic forgetting.

3.4.3 Optimization and Convergence

Optimizer: Adam with standard hyperparameters (learning rate 1e-4, weight decay 1e-6).

Learning Rate Schedule: Cosine annealing with 160,000 training iterations (on 376,000 masks, $\sim$0.4 epochs over the training portion of SA-1B used at the time of publication).

Batch Size: 256 (distributed across 64 TPU-v4 cores).

Training Duration: $\sim$3 days on 64 TPU-v4 cores.

3.5 Inference and Computational Efficiency

3.5.1 Two-Stage Inference

Stage 1: Image Encoding (Offline, Amortized)

- Input: Image ($H \times W$ pixels).
- Process: Feed through MAE pre-trained ViT-H.
- Output: Image Embedding $(H/16) \times (W/16) \times 1280$ (for 16-pixel patches).

- Latency: ∼50–100ms on GPU, seconds on CPU (negligible, often done offline).
- **Amortization:** This step is performed once per image, regardless of the number of prompts.

Stage 2: Prompt Encoding + Mask Decoding (Real-Time, Per-Prompt)

- Input: Cached Image Embedding + Prompt (point, box, text, or mask).
- Process: Prompt Encoder → Two-way Transformer Decoder.
- Output: 3 Masks + 3 IoU scores.
- Latency: ∼50ms in web browser on CPU (no GPU required).
- **Per-Prompt Amortization:** Fixed cost regardless of image resolution.

3.5.2 Web Browser Efficiency

SAM's ability to run in a web browser with 50ms latency per prompt is remarkable and achieved through careful design:

1. **Small Model Components:** Prompt Encoder and Mask Decoder are lightweight (<3% of total model parameters).
2. **CPU Compatibility:** No CUDA-specific optimizations; uses standard WebGL/WebAssembly for browser computation.
3. **Cached Computation:** Image embedding is computed once (potentially on the server and sent to the browser) and reused.

3.6 Summary: The Unified System

SAM represents a unified solution to the foundation model problem for segmentation:

1. **Task (Promptable Segmentation):** Trained on a diverse, billion-scale dataset using visual prompts.
2. **Model (Efficient Decoupled Architecture):** Separates expensive image encoding (once) from lightweight prompt decoding (amortized).
3. **Data Engine (Flywheel Effect):** Iteratively improved the model to enable increasingly automated data collection.
4. **Ambiguity Handling (Multiple Outputs):** Predicts three valid masks per prompt, handling real-world segmentation ambiguity.
5. **Zero-Shot Transfer:** Achieves strong performance on diverse downstream tasks without fine-tuning via prompt engineering.

This design philosophy—decoupled computation, amortized inference, and multi-modal prompting—sets a new paradigm for interactive and automated segmentation systems.

Generate two "corner points": $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$.

Chapter 4

Implementation and Experiment

To evaluate the practical capabilities and performance of the Segment Anything Model (SAM), this study utilizes the official implementation provided by Meta AI Research. The core architecture remains consistent with the original proposal; however, the codebase was streamlined by removing extraneous files and modules not required for this specific demonstration. The complete source code and configuration files used in these experiments are available at the following repository: `https://github.com/facebookresearch/segment-anything`.

4.1 Experimental Environment

The experiments were conducted in a **Python 3.11** environment. To ensure the smooth execution of the Segment Anything Model and the evaluation of the results, the following core libraries and frameworks were utilized:

- **Deep Learning Frameworks:** `torch` and `torchvision` for model loading and inference.
- **Optimization and Deployment:** `onnx`, `onnxruntime`, and `onnxscript` for model exportation and runtime acceleration.

- **Image Processing and Data Handling:** `opencv-python` for real-time computer vision tasks and `pycocotools` for performance metric calculations.
- **Visualization and Development:** `matplotlib` for generating result plots and `jupyter` for interactive experimentation.

4.2 Model Checkpoints and Demonstration Modes

Due to storage constraints, model weights are not included directly in the repository. Users are encouraged to download the pre-trained checkpoints as specified in the `Source_code/README.md` file. The implementation supports three distinct Vision Transformer (ViT) backbones: **SAM-H** (`vit_h`), **SAM-L** (`vit_l`), and **SAM-B** (`vit_b`).

The demonstration is structured into three primary interfaces:

1. **Interactive Notebooks:** Located in `Source_code/notebooks/`, these provide a guided environment for manual prompting and automatic mask generation. Users can follow the inline instructions within `predictor_example.ipynb` and `automatic_mask_generator_example.ipynb`.
2. **Command Line Interface (CLI):** For batch processing, a Python script is provided. After navigating to the `Source_code` directory, the following command structure is used:

```
1    python scripts/amg.py --checkpoint <path/to
   /checkpoint> --model-type <model_type> --
   input <image_or_folder> --output <path/to/
   output>
2
```

3. **Web-Based Application:** A specialized demo requiring **Node.js** and **Yarn**. This mode involves a two-step process: exporting image embeddings and the ONNX model before building the static web application. Detailed setup steps are found in `Source_code/demo/README.md`.

4.2.1 Pre-processing for Web Demonstration

To achieve real-time interactivity within the web-based demonstration, a decoupled inference strategy is employed. This requires migrating the model from a heavy Python environment to a lightweight client-side runtime via the following pre-processing steps:

ONNX Model Export

The SAM mask decoder must be exported to the ONNX format to optimize inference performance in the browser. This can be accomplished through two methods:

- **Interactive Export:** Using the `Source_code/notebooks/onnx_model_example.ipynb` notebook for a guided, step-by-step export process.
- **Script-Based Export:** Executing the automated script from the `Source_code` directory:

```
1    python scripts/export_onnx_model.py --
   checkpoint <path/to/checkpoint> --model-type
    <model_type> --output <path/to/output>
2
```

Image Embedding Extraction

Since the Vision Transformer (ViT) image encoder is computationally expensive, the web demo does not run it locally in the browser. Instead,

users must pre-calculate the high-dimensional image embeddings for each target image. This extraction is performed using the utilities provided in `Source_code/notebooks/onnx_model_example.ipynb`.

Deployment to Web Frontend

For the static web application to function, the generated outputs must be manually integrated into the frontend assets. Users must copy the exported ONNX model (`.onnx`) into `Source_code/demo/model` and the precomputed image embeddings (`.npy`) into `Source_code/demo/src/assets/data`. This architecture allows the web app to perform a simple "lookup" of image features, enabling the SAM mask decoder to generate segments in response to user clicks with millisecond latency.

4.3 Model Architecture

The Segment Anything Model (SAM) inference pipeline consists of three primary stages: image encoding, prompt encoding, and mask decoding. This section details the architectural components implemented in the core codebase.

4.3.1 Common Architectural Blocks - common.py

The model utilizes foundational layers defined in `common.py` to maintain consistency across its sub-networks:

- **MLPBlock:** A standard Feed-Forward Network (FFN) that utilizes two linear layers with a GELU activation function. It projects input embeddings into a higher-dimensional space (`mlp_dim`) before restoring the original embedding dimension.
- **LayerNorm2d:** A specialized normalization layer designed for 2D spatial data. Unlike standard `LayerNorm`, which typically operates

on the last dimension, this implementation calculates mean and variance across the channel dimension while preserving spatial height and width. This is essential for maintaining feature map integrity in convolutional "neck" modules.

4.3.2 Image Encoder - image_encoder.py

The `ImageEncoderViT` serves as the backbone of the system, transforming raw imagery into high-dimensional feature embeddings. The architectural flow and corresponding tensor transformations are summarized below:

1. **Initial Projection:** The input image $x : [B, C_{in}, H_{img}, W_{img}]$ (e.g., $[B, 3, 1024, 1024]$) is processed by the `PatchEmbed` module. This utilizes a convolutional layer with a 16×16 kernel and stride, divides the image into patches and an convolutional layer is applied, yielding a tensor of shape $[B, H_p, W_p, D_{embed}]$ (e.g., $[B, 64, 64, 768]$).

2. **Positional Encoding:** To provide the transformer blocks with spatial context, the model integrates absolute positional embeddings. When `use_abs_pos` is enabled, the parameter `self.pos_embed` is initialized as a learnable tensor of zeros with the shape $[1, H_p, W_p, D_{embed}]$ (e.g., $[1, 64, 64, 768]$). During the forward pass, this spatial information is injected into the patch tokens via element-wise addition: $x = x + \text{self.pos\_embed}$.

   This zero-initialization strategy is a standard practice in Vision Transformer architectures, such as ViTDet, to avoid introducing stochastic noise during the initial stages of training. Since the self-attention mechanism is inherently permutation-invariant, it treats patches as an unordered set. By utilizing a learnable parameter starting from zero, the model can adaptively learn optimal spatial encodings tailored specifically for dense segmentation tasks without an initial po-

sitional bias. Furthermore, this parameter acts as a placeholder that is typically overwritten by learned values when loading pre-trained checkpoints from large-scale datasets like SA-1B.

3. **Transformer Processing:** The data passes through a sequence of **12 `Block` modules**, where each block functions as a transformer encoder. Within these blocks:

    - Each encoder utilizes **12 multi-attention heads** to process the embeddings.
    - The attention mechanism can be configured for either global attention or window-based attention to optimize computational efficiency.
    - Each block applies `LayerNorm` before the attention layer and again before a `MLPBlock`, utilizing residual connections to facilitate gradient flow.

4. **Neck and Final Embedding:** To prepare the features for the mask decoder, the tensor is permuted to a channel-first format $[B, D_{embed}, H_p, W_p]$. It then passes through a "neck" consisting of a 1×1 convolution, `LayerNorm2d`, a 3×3 convolution, and a final `LayerNorm2d` to refine the feature maps.

5. **Output:** The final image embedding is produced with shape $[B, C_{out}, H_p, W_p]$ (e.g., $[B, 256, 64, 64]$), representing the extracted features ready for downstream prompting.

4.3.3 Prompt Encoder - prompt_encoder.py

The `PromptEncoder` is responsible for translating various user-defined prompts—such as points, bounding boxes, and low-resolution masks—into

a unified embedding space compatible with the mask decoder. It categorizes inputs into sparse and dense representations to maintain architectural flexibility.

Sparse and Dense Representations

The `PromptEncoder` processes multi-modal inputs through specialized pipelines determined by their geometric properties, mapping them into a unified embedding space for the mask decoder.

- **Point Prompts:** Input coordinates, provided as a tensor of shape $[B, N_{points}, 2]$, are normalized relative to the input image size. These coordinates are transformed into positional encodings via random spatial frequencies in the `PositionEmbeddingRandom` layer. Each point embedding is then augmented by a learnable vector that identifies the point as either a foreground or background prompt. The specific embedding is selected based on a corresponding label tensor of shape $[B, N_{points}]$.

- **Box Prompts:** Bounding boxes are modeled as coordinate pairs representing the top-left and bottom-right corners. The input of shape $[B, 4]$ is reshaped to $[B, 2, 2]$ and encoded using the same positional encoding logic as point prompts. To distinguish spatial boundaries, specific learnable corner embeddings are added to the corner encodings. For the purposes of this implementation and to ensure clarity in demonstration, the model currently supports one box prompt per image.

- **Mask Prompts:** Dense input masks with an initial shape $[B, 1, H_{img}, W_{img}]$ are first interpolated to the required resolution before being processed by the `mask_downscaling` block. This block consists of a sequence of two 2×2 convolutions (stride 2) and a final

1×1 convolution, interspersed with `LayerNorm2d` and GELU activations. This process reduces the spatial resolution by a factor of 4 to match the image embedding (H_p, W_p) while increasing the channel depth to `embed_dim` (standardly 256).

Positional Encoding Logic

Spatial awareness is injected via the `PositionEmbeddingRandom` class, which utilizes a fixed Gaussian matrix to project coordinates into a high-dimensional space.

1. **Sparse Transformation:** Coordinates are mapped to a sparse embedding of shape $[B, N, D_{embed}]$, where N denotes the number of input prompts and D_{embed} is the embedding dimension (standardly 256).

2. **Dense Transformation:** For mask inputs of shape $[B, 1, H_{mask}, W_{mask}]$, the downscaling operation produces a dense embedding of shape $[B, D_{embed}, H_p, W_p]$ (e.g., $[B, 256, 64, 64]$).

3. **Global Reference:** The module provides a dense positional encoding for the entire image grid via `get_dense_pe()`, yielding a tensor of shape $[1, D_{embed}, H_p, W_p]$.

If no mask prompt is provided, the model utilizes a learnable `no_mask_embed` to act as a placeholder, ensuring the mask decoder always receives a consistent tensor shape for the dense prompt input.

4.3.4 Two-Way Transformer - transformer.py

The `TwoWayTransformer` serves as the core reasoning engine within the mask decoder. It is designed to facilitate a "two-way" interaction between the sparse prompt embeddings (queries) and the dense image embeddings (keys).

Architectural Design

As implemented in `transformer.py`, the module consists of a series of `TwoWayAttentionBlock` layers followed by a final attention operation:

- **Layer Composition:** The transformer consists of a specified `depth` (number of layers), an `embedding_dim` for channel consistency, and a multi-head attention mechanism with `num_heads`.
- **Input Transformation:** Before processing, the 2D image embeddings of shape $[B, C, H, W]$ are flattened and permuted into a sequence format of shape $[B, H \times W, C]$ to represent image tokens.

The Two-Way Attention Block

Each block in the transformer performs four distinct operations to refine the relationship between prompts and the image:

1. **Sparse Self-Attention:** The prompt tokens perform self-attention to aggregate information between multiple prompts (e.g., multiple points or box corners).
2. **Token-to-Image Cross-Attention:** Prompt tokens (queries) attend to the image embeddings (keys) to extract relevant spatial features.
3. **MLP Block:** A point-wise `MLPBlock` is applied to the prompt tokens for feature transformation and projection.
4. **Image-to-Token Cross-Attention:** In the "two-way" step, the image embeddings (now acting as queries) attend back to the prompt tokens to update the dense feature map based on the prompt locations.

Final Attention and Output

After the repeated blocks, the transformer performs a final attention pass where prompt tokens attend to the refined image embeddings.

- **Positional Embedding Integration:** Throughout all attention steps, positional encodings for both the image (`image_pe`) and prompts (`point_embedding`) are added to the queries and keys to maintain spatial awareness.
- **Final Normalization:** The resulting prompt tokens are passed through a final `LayerNorm` before being returned to the mask decoder for final logit prediction.

Component	Input Type	Role in SAM
`self_attn`	Sparse Tokens	Inter-prompt communication
`cross_attn_token_to_image`	Sparse $\rightarrow$ Dense	Prompt-driven feature extraction
`mlp`	Sparse Tokens	Feature projection and non-linearity
`cross_attn_image_to_token`	Dense $\rightarrow$ Sparse	Updating image features via prompts

Table 4.1: Functional layers within the Two-Way Attention Block.

4.3.5 Mask Decoder - mask_decoder.py

The `MaskDecoder` is the final stage of the SAM architecture, responsible for translating the processed image features and prompt embeddings into specific segmentation masks and corresponding quality scores. It utilizes a transformer-based architecture to map the interaction between image and prompt data.

Architectural Components

The decoder integrates several specialized components to generate high-fidelity outputs:

- **Output Tokens:** The model utilizes learnable embeddings, including an `iou_token` to predict mask quality and a set of `mask_tokens` used to handle mask disambiguation.
- **Two-Way Transformer:** A transformer module facilitates cross-attention between the image embeddings and the prompt tokens, refining both sets of features simultaneously.
- **Output Upscaling:** To restore spatial resolution, the decoder employs a sequence consisting of two transposed convolutions and `LayerNorm2d` layers. This blocks upscales the feature maps from the transformer back toward the original input resolution.
- **Dynamic Hypernetworks:** Instead of static layers, the model uses `output_hypernetworks_mlps` to transform the refined mask tokens into weights for a final linear layer that predicts the masks.

Forward Pass and Tensor Transformations

The `MaskDecoder` manages a complex flow of data to produce final predictions:

1. **Token Concatenation:** The `iou_token` and `mask_tokens` are concatenated with the `sparse_prompt_embeddings` to form the initial token set for the transformer.
2. **Feature Integration:** Image embeddings from the encoder are combined with `dense_prompt_embeddings` and the image positional encodings (`image_pe`).

3. **Transformer Execution:** The transformer processes these inputs, outputting refined hidden states for the tokens and an updated feature map.

4. **Mask Generation:**

    - The updated image features are upscaled to a shape of $[B, D_{embed}/8, H_p \times 4, W_p \times 4]$ (e.g., $[B, 32, 256, 256]$).
    - Mask tokens are passed through the hypernetwork MLPs to produce mask coefficients.
    - Final masks are generated via a spatially point-wise product between these coefficients and the upscaled embeddings.

5. **Quality Prediction:** The `iou_token` output is passed through the `iou_prediction_head` (an MLP) to generate an Intersection-over-Union (IoU) score, estimating the quality of each predicted mask.

4.3.6 System Integration and End-to-End Inference - sam.py

The `Sam` class serves as the primary orchestrator that integrates the image encoder, prompt encoder, and mask decoder into a unified pipeline. It manages the full inference lifecycle, from raw image pre-processing to the generation of final binary masks at the original image resolution.

Pre-processing and Normalization

Before feature extraction, input images undergo a standardization process within the `preprocess` method to ensure compatibility with the Vision Transformer backbone:

- **Color Normalization:** Pixel values are normalized using the specific mean ($[123.675, 116.28, 103.53]$) and standard deviation ($[58.395, 57.12, 57.375]$) constants registered in the model's buffers.
- **Spatial Padding:** Images are padded to a square format matching the encoder's `img_size` (standardly 1024) to maintain consistent patch dimensions and spatial alignment.

Post-processing and Mask Refinement

The raw outputs from the mask decoder are low-resolution logits (256×256). To produce the final output, the `postprocess_masks` function performs the following operations:

- **Rescaling:** The logits are bilinearly interpolated back to the model's internal input resolution (1024×1024).
- **Padding Removal:** Any padding introduced during the pre-processing stage is cropped based on the `input_size` to restore the original aspect ratio.
- **Final Restoration:** The masks are interpolated to the image's `original_size` and thresholded against `mask_threshold` (defaulting to 0.0) to produce the final binary segmentation maps.

Architecture Summary Table

The following table summarizes the primary tensor transformations across the Segment Anything Model pipeline.

Stage	Input Shape	Output Shape
Image Encoder (ViT)	$[B, 3, 1024, 1024]$	$[B, 256, 64, 64]$
Prompt Encoder (Sparse)	Coordinates/Labels	$[B, N, 256]$
Prompt Encoder (Dense)	$[B, 1, 256, 256]$	$[B, 256, 64, 64]$
Mask Decoder (Logits)	Combined Embeddings	$[B, C, 256, 256]$
Final Prediction	$[B, C, 256, 256]$	$[B, C, H_{orig}, W_{orig}]$

Table 4.2: Tensor shapes and data flow through the SAM architecture.

This architecture allows the model to output multiple masks (typically three) for a single prompt, ensuring robustness in ambiguous scenarios where a point or box might refer to multiple valid sub-parts of an object.

4.4 SAM Predictor - predictor.py

The `SamPredictor` class provides a high-level interface for using the Segment Anything Model to predict masks based on user-defined prompts. It is specifically designed to handle the initial heavy computation of image embeddings once, allowing for subsequent, efficient real-time mask prediction with varying prompts.

Image Pre-processing and Embedding

Before prediction, an image must be registered using the `set_image` method, which executes the following pipeline:

- **Color Format Adjustment:** The input image, expected as an `HWC` `uint8` array, is converted to the model's internal RGB format if necessary.
- **Spatial Transformation:** The image is resized using the `ResizeLongestSide` utility to ensure its longest dimension matches the image encoder's expected input size (standardly 1024).

Feature Extraction: The transformed image is passed through the `ImageEncoderViT` to calculate the image embeddings, which are then cached within the predictor as the `features` attribute.

Prompt Processing and Mask Prediction

Once the image is set, the `predict` method facilitates mask generation by processing point, box, or mask prompts:

1. **Prompt Transformation:** Input coordinates (points and boxes) are transformed from the original image coordinate frame to the model's resized input frame.

2. **Prompt Embedding:** These transformed prompts are passed to the `PromptEncoder` to generate sparse and dense embeddings.

3. **Decoding:** The cached image embeddings and the new prompt embeddings are processed by the `MaskDecoder` to generate low-resolution mask logits and quality scores.

4. **Post-processing:** The logits are upscaled to the original image resolution using bilinear interpolation and thresholded to produce binary masks.

Multi-mask Output and Refinement

A key feature of the predictor is the `multimask_output` toggle. When enabled, the model returns three distinct masks for each prompt, which is particularly effective for resolving ambiguity in sparse inputs, such as a single point click. Furthermore, the predictor returns low-resolution logits (256×256) that can be fed back into the model as a `mask_input` for iterative mask refinement.

Predictor Method	Function
`set_image`	Calculates and caches image embeddings for repeated use
`predict`	Predicts masks for NumPy inputs and handles coordinate scaling
`predict_torch`	Core prediction method taking pre-transformed batched tensors
`reset_image`	Clears cached embeddings and image metadata

Table 4.3: Key functional methods of the `SamPredictor` class.

4.5 Automatic Mask Generator - automatic_mask_generator.py

The `SamAutomaticMaskGenerator` class implements the "Everything" mode of the Segment Anything Model, enabling the generation of masks for an entire image without manual prompting. It achieves this by sampling a systematic grid of point prompts and filtering the resulting predictions based on quality and redundancy.

Point Selection and Crop Strategy

To ensure total coverage of the image, the generator employs a multi-layered sampling approach:

- **Point Grids:** By default, the model samples a grid of 32×32 points across the image. For higher resolution or complex scenes, the `crop_n_layers` parameter allows the model to run again on overlapping image crops, increasing the density of point prompts in localized regions.
- **Batch Processing:** To optimize GPU utilization, points are processed in batches (defaulting to 64 points per batch) rather than individually.

Filtering and Quality Control

Since a dense grid of points generates many overlapping or low-quality masks, the following filtering pipeline is applied:

1. **Predicted IoU Threshold:** Masks are discarded if the model's own quality prediction (`predicted_iou`) falls below a set threshold (standardly 0.88).
2. **Stability Score:** The `calculate_stability_score` function measures how much a mask changes when the binarization threshold is shifted. Masks that are highly sensitive to these shifts are considered unstable and are filtered out.
3. **Boundary Filtering:** Boxes that touch the edge of an image crop are often incomplete and are removed to prevent artifacts.
4. **Non-Maximal Suppression (NMS):** To resolve duplicate predictions, NMS is performed twice—first within individual crops (`box_nms_thresh`) and subsequently across different crop layers (`crop_nms_thresh`).

Post-processing

The final stage of the automatic pipeline involves refining the geometric properties of the generated segments:

- **Small Region Removal:** Utilizing OpenCV, the model can remove disconnected fragments or fill small holes within masks if their area is below the `min_mask_region_area`.
- **Output Encoding:** To manage memory efficiency for high-resolution images, masks can be returned as Run-Length Encodings (RLE) or COCO-compatible RLEs instead of raw binary arrays.

Parameter	Default Value	Function
`points_per_side`	32	Density of the initial point grid
`pred_iou_thresh`	0.88	Minimum predicted mask quality
`stability_score_thresh`	0.95	Filter for binarization stability
`box_nms_thresh`	0.70	Threshold for duplicate removal

Table 4.4: Key configuration parameters for the Automatic Mask Generator.

Chapter 5

Kết luận và hướng phát triển

5.1 Conclusion

5.2 Discussion

5.2.1 Limitations

5.2.2 Future Work

“ [7]

Tài liệu tham khảo

[1] Alexander Kirillov et al. "Segment anything". In: *arXiv preprint arXiv:2304.026*
(2023).

[2] Nikhila Ravi et al. "SAM 2: Segment Anything in Images and Videos".
In: *arXiv preprint arXiv:2408.00714* (2024).